Neovim

Vim essentials plus modern Lua, LSP, terminal, and runtime tooling

VIM-STYLE CORE

`nvim <file> / nvim .`
Edit file / start in directory.

`i / Esc / :wq`
Insert / Normal / save+exit.

`gg / G / 42G`
First / last / line 42.

`ciw / di( / yap`
Change word / delete in () / yank paragraph.

`/term / n / N`
Search and cycle matches.

`:sp / :vsp / Ctrl-w w`
Split horizontally/vertically; cycle windows.

`:bnext / :bd`
Next / close buffer.

`:help lua-guide`
Read Lua configuration help.

LUA API AND KEYMAPS

`vim.opt.number = true`
Show line numbers.

`vim.opt.relativenumber = true`
Enable relative line numbers.

`vim.g.mapleader = " "`
Set leader before creating mappings.

`vim.keymap.set("n", lhs, rhs)`
Map normal-mode input to keys/function.

`vim.cmd("edit file.c")`
Execute Ex commands from Lua.

`vim.fn.expand("%:p")`
Use Vimscript function in Lua.

`vim.api.nvim_create_autocmd(...)`
Register an event-driven hook.

`vim.notify("message")`
Display notification.

HEALTH AND CONFIGURATION

`:checkhealth`
Diagnose Neovim installation and providers.

`:checkhealth vim.lsp`
Check language-server configuration.

`:lua =vim.fn.stdpath("config")`
Print configuration directory.

`~/.config/nvim/init.lua`
Typical main configuration on macOS/Linux.

`:source $MYVIMRC`
Reload current config (some effects persist).

`:messages`
Inspect messages and error output.

`:verbose map <keys>`
Find where mapping is defined.

BUILT-IN TERMINAL

`:terminal`
Open terminal buffer.

`i`
Enter Terminal input from Normal mode.

`Ctrl-\ Ctrl-n`
Leave Terminal input; return to Normal.

`Ctrl-w h/j/k/l`
Navigate among editor windows.

`:terminal make test`
Run shell command in terminal buffer.

`nvim --headless +"q"`
Run without terminal UI.

BUILT-IN LSP (0.11+)

Neovim

Vim essentials plus modern Lua, LSP, terminal, and runtime tooling

`vim.lsp.config("clangd", {...})`
Configure a language server.

`vim.lsp.enable("clangd")`
Enable clangd config; install server separately.

`vim.lsp.buf.definition()`
Jump to symbol definition.

`vim.lsp.buf.references()`
List references.

`vim.lsp.buf.rename()`
Rename symbol.

`vim.lsp.buf.code_action()`
Get code actions.

`vim.lsp.buf.format({async=true})`
Ask language server to format buffer.

`vim.diagnostic.open_float()`
Show diagnostic in floating window.

`:checkhealth vim.lsp`
Diagnose active servers.

PARSERS, PLUGINS, RUNTIME

`:Inspect`
Inspect syntax under cursor.

`vim.treesitter.start()`
Begin tree-sitter highlighting; parser needed.

`:packadd <name>`
Load optional Vim package.

`vim.pack.add({...})`
Built-in plugin manager on supporting versions.

`:packupdate`
Update vim.pack-managed plugins (version dependent).

`vim.fn.has("nvim-0.11")`
Guard version-specific code.

`vim.api.nvim_buf_get_lines(...)`
Read buffer lines from Lua.

KEY CONSIDERATIONS

LSP CLIENT IS BUILT IN; SERVERS ARE NOT. Install clangd or other servers separately. Plugin shortcuts such as <leader>ff are not universal defaults. Neovim 0.11 introduced the vim.lsp.config()/enable() API; check version before use.

OFFICIAL DOCUMENTATION

`https://neovim.io/doc/user/lua-guide/`

`https://neovim.io/doc/user/lsp/`

`https://neovim.io/doc/user/pack/`